

Ai assisted coding Assignment-21.5

Name:N.vaishnavi

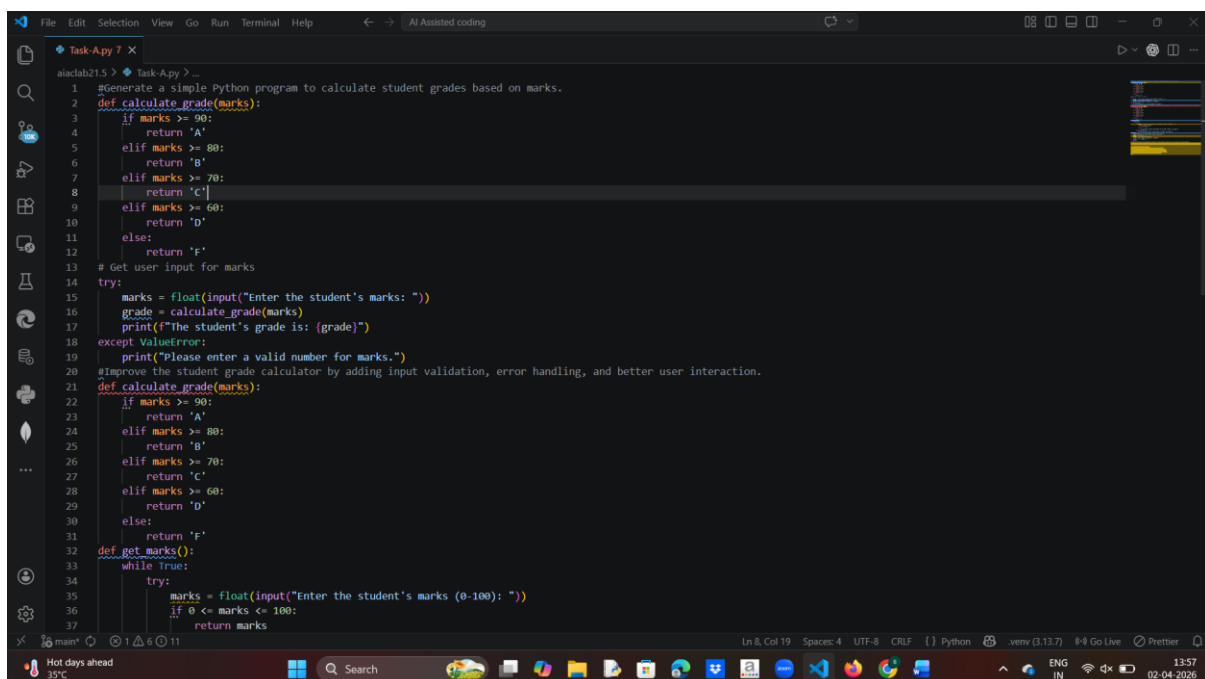
Batch-44

HTNO:2303A52405S

Task A – AI-Based Initial Program Development

Prompt: 1. Generate a simple Python program to calculate student grades based on marks.

2. Improve the student grade calculator by adding input validation, error handling, and better user interaction.



```
1 #Generate a simple Python program to calculate student grades based on marks.
2 def calculate_grade(marks):
3     if marks >= 90:
4         return 'A'
5     elif marks >= 80:
6         return 'B'
7     elif marks >= 70:
8         return 'C'
9     elif marks >= 60:
10        return 'D'
11    else:
12        return 'F'
13    # Get user input for marks
14    try:
15        marks = float(input("Enter the student's marks: "))
16        grade = calculate_grade(marks)
17        print(f"The student's grade is: {grade}")
18    except ValueError:
19        print("Please enter a valid number for marks.")
20    #Improve the student grade calculator by adding input validation, error handling, and better user interaction.
21    def calculate_grade(marks):
22        if marks >= 90:
23            return 'A'
24        elif marks >= 80:
25            return 'B'
26        elif marks >= 70:
27            return 'C'
28        elif marks >= 60:
29            return 'D'
30        else:
31            return 'F'
32    def get_marks():
33        while True:
34            try:
35                marks = float(input("Enter the student's marks (0-100): "))
36                if 0 <= marks <= 100:
37                    return marks
```

```
21 def calculate_grade(marks):
24     elif marks >= 80:
25         return 'B'
26     elif marks >= 70:
27         return 'C'
28     elif marks >= 60:
29         return 'D'
30     else:
31         return 'F'
32 def get_marks():
33     while True:
34         try:
35             marks = float(input("Enter the student's marks (0-100): "))
36             if 0 <= marks <= 100:
37                 return marks
38             else:
39                 print("Marks should be between 0 and 100. Please try again.")
40         except ValueError:
41             print("Invalid input. Please enter a number for marks.")
42 def main():
43     print("Welcome to the Student Grade Calculator!")
44     marks = get_marks()
45     grade = calculate_grade(marks)
46     print(f"The student's grade is: {grade}")
47 if __name__ == "__main__":
48     main()
49
50 #Compare the basic and enhanced versions of the grade calculator and explain the improvements.
51 '''The basic version of the grade calculator simply takes user input for marks and calculates the grade without any validation or error handling.
52 It assumes that the user will always enter valid input, which can lead to issues if the user enters non-numeric values or marks outside the valid range (0-100).
53
54 The enhanced version includes:
55 1. Input validation to ensure marks are within 0-100
56 2. Error handling with try-except for non-numeric inputs
57 3. A separate function for getting marks with retry logic
58 4. Better user interaction with clear prompts and messages
59 5. A main() function for better code organization'''
```

```
PS C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding> & "C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding\.venv\Scripts\activate.ps1"
(.venv) PS C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding> & "C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding\.venv\Scripts\python.exe" "C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding\aiacalab21.5\Task-A.py"
Enter the student's marks: 70
The student's grade is: C
Welcome to the Student Grade Calculator!
Enter the student's marks (0-100): 90
The student's grade is: A
(.venv) PS C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding>
```

The basic version of the grade calculator simply takes user input for marks and calculates the grade without any validation or error handling.

It assumes that the user will always enter valid input, which can lead to issues if the user enters

non-numeric values or marks outside the valid range (0-100).

The enhanced version includes:

1. Input validation to ensure marks are within 0-100
2. Error handling with try-except for non-numeric inputs
3. A separate function for getting marks with retry logic
4. Better user interaction with clear prompts and messages
5. A main() function for better code organization

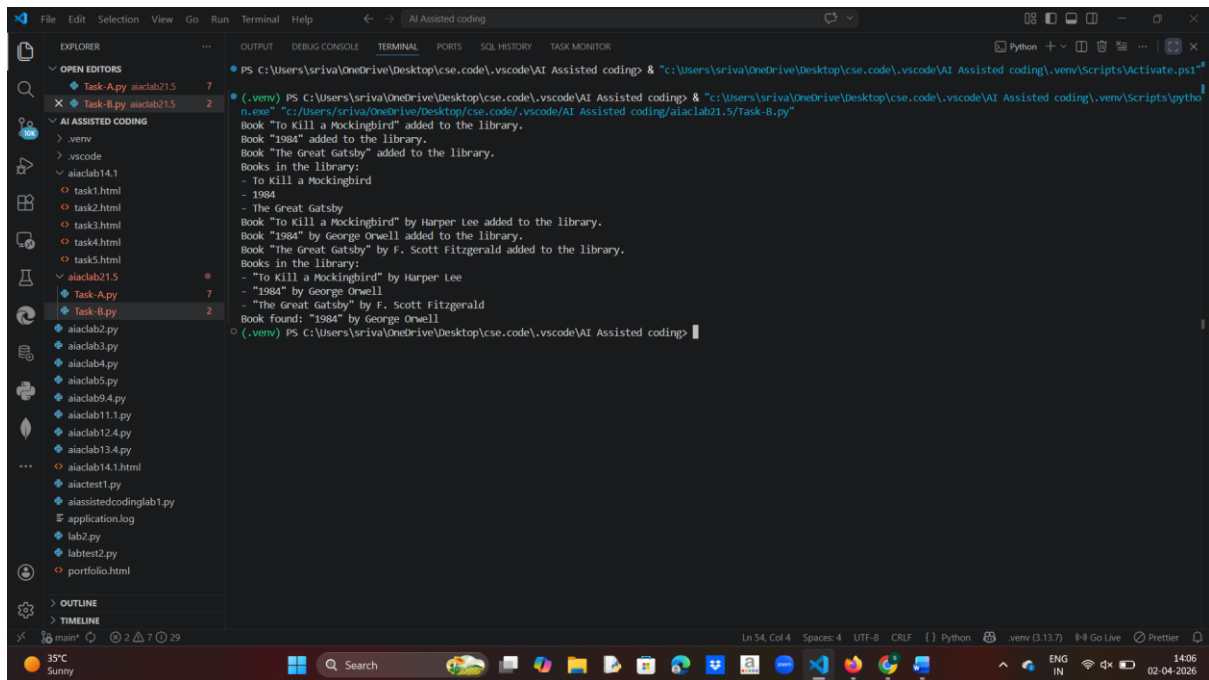
Task B – AI-Assisted Application Improvement

Prompt: 1. Generate a simple Python program for a basic library management system.

2. Enhance the library management system by adding features like search, display, and better data structure.

```
1 #Generate a simple Python program for a basic library management system.
2 class Library:
3     def __init__(self):
4         self.books = []
5     def add_book(self, title):
6         self.books.append(title)
7         print(f'Book "{title}" added to the library.')
8     def display_books(self):
9         if self.books:
10             print("Books in the library:")
11             for book in self.books:
12                 print(f"- {book}")
13         else:
14             print("No books in the library.")
15 # Create a library instance and add some books
16 library = Library()
17 library.add_book("To Kill a Mockingbird")
18 library.add_book("1984")
19 library.add_book("The Great Gatsby")
20 library.display_books()
21 #Enhance the library management system by adding features like search, display, and better data structure.
22 class Library:
23     def __init__(self):
24         self.books = {}
25     def add_book(self, title, author):
26         self.books[title] = author
27         print(f'Book "{title}" by {author} added to the library.')
28     def display_books(self):
29         if self.books:
30             print("Books in the library:")
31             for title, author in self.books.items():
32                 print(f"- {title} by {author}")
33         else:
34             print("No books in the library.")
35     def search_book(self, title):
36         if title in self.books:
37             print(f'Book found: "{title}" by {self.books[title]}')
```

```
19 library.add_book("The Great Gatsby")
20 library.display_books()
21 #Enhance the library management system by adding features like search, display, and better data structure.
22 class Library:
23     def __init__(self):
24         self.books = {}
25     def add_book(self, title, author):
26         self.books[title] = author
27         print(f'Book "{title}" by {author} added to the library.')
28     def display_books(self):
29         if self.books:
30             print("Books in the library:")
31             for title, author in self.books.items():
32                 print(f"- {title} by {author}")
33         else:
34             print("No books in the library.")
35     def search_book(self, title):
36         if title in self.books:
37             print(f'Book found: "{title}" by {self.books[title]}')
38         else:
39             print(f'Book "{title}" not found in the library.')
40 # Create a library instance and add some books
41 library = Library()
42 library.add_book("To Kill a Mockingbird", "Harper Lee")
43 library.add_book("1984", "George Orwell")
44 library.add_book("The Great Gatsby", "F. Scott Fitzgerald")
45 library.display_books()
46 library.search_book("1984")
47 analyze how the improved version is better in terms of structure and performance.
48 """The improved version of the library management system has several enhancements over the basic version:
49 1. Data Structure: The improved version uses a dictionary to store books, allowing for more efficient lookups and better organization of book information.
50 2. Search Functionality: The improved version includes a search function that allows users to find books by title, which is a common feature in library systems.
51 3. Better User Interaction: The improved version provides clearer prompts and messages, enhancing the user experience.
52 4. Code Organization: The improved version is more structured, with separate methods for adding books, displaying books, and searching for books, making
53 Overall, the improved version is more functional, user-friendly, and efficient compared to the basic version.
54 """
```

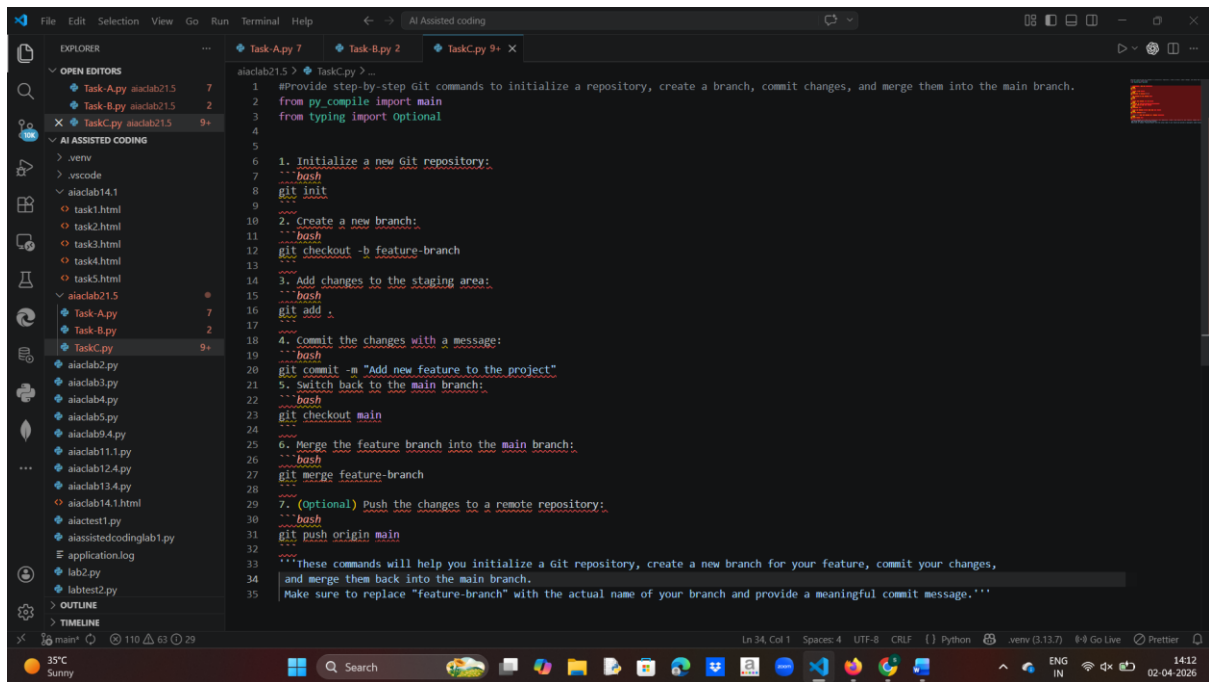


3. Improvements

- Structured data (dictionary)
- Search functionality
- Better scalability

Task C – Git Workflow Practice

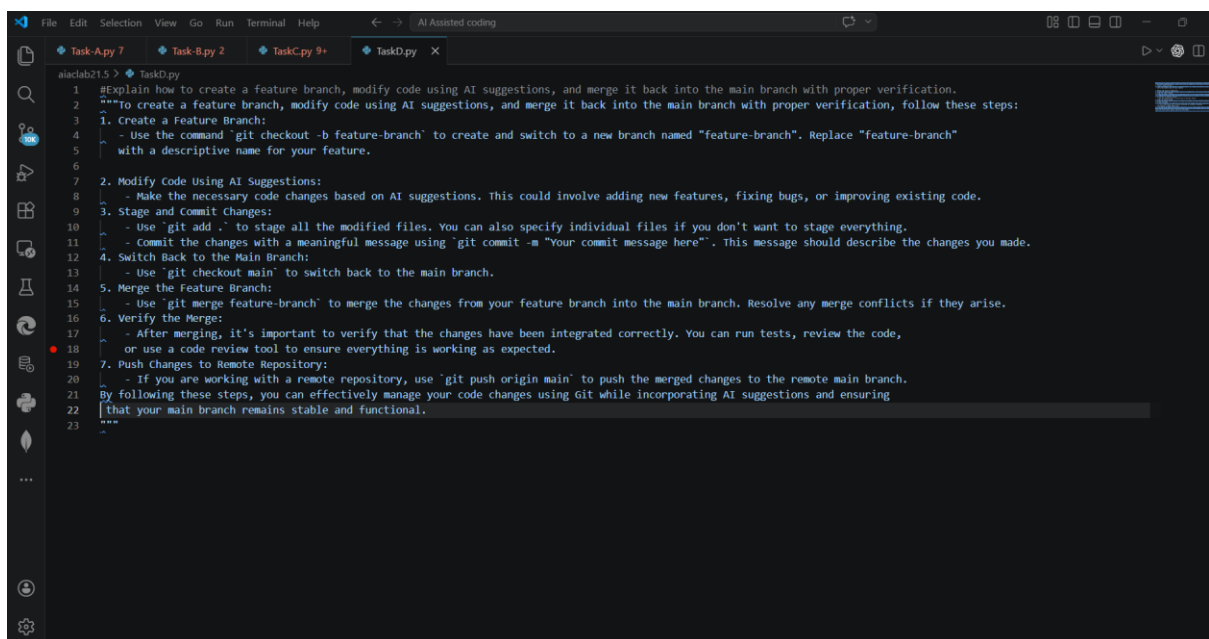
Prompt: Provide step-by-step Git commands to initialize a repository, create a branch, commit changes, and merge them into the main branch.



Task D – Branching and Merging with AI Support

Prompt:

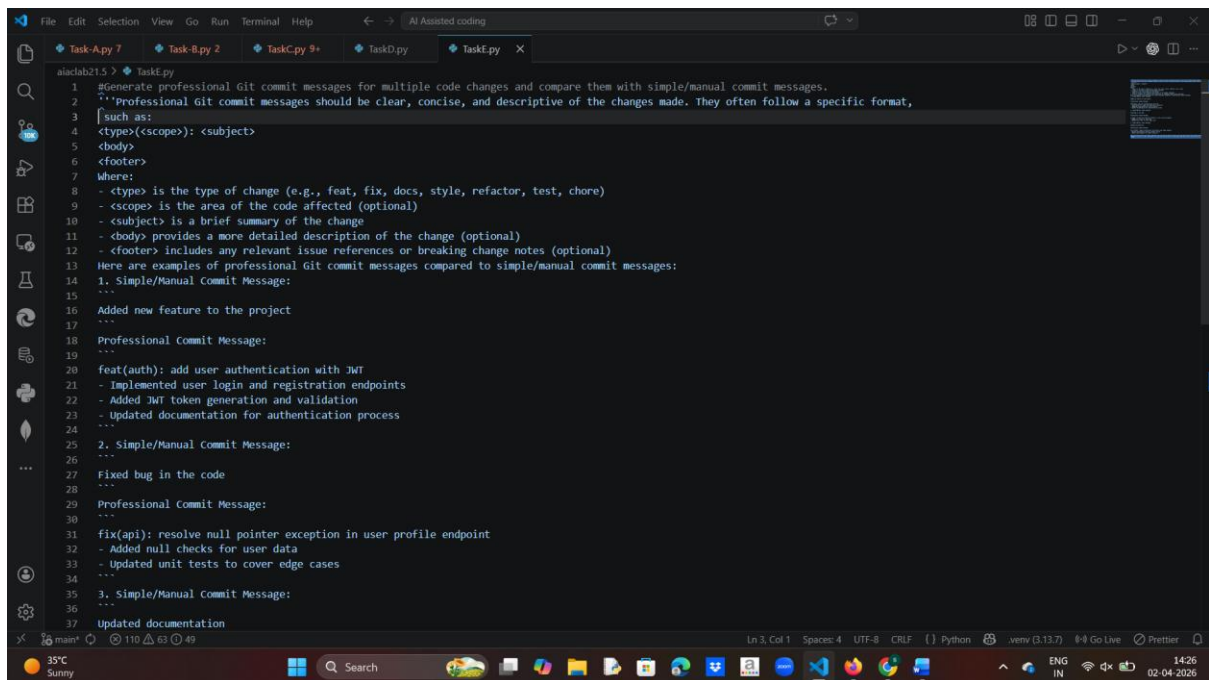
Explain how to create a feature branch, modify code using AI suggestions, and merge it back into the main branch with proper verification.



Task E – AI for Commit Message Writing

Prompt:

Generate professional Git commit messages for multiple code changes and compare them with simple/manual commit messages.



```
1 Generate professional Git commit messages for multiple code changes and compare them with simple/manual commit messages.
2 """Professional git commit messages should be clear, concise, and descriptive of the changes made. They often follow a specific format,
3 such as:
4 <type><scope>: <subject>
5 <body>
6 <footer>
7 Where:
8 - <type> is the type of change (e.g., feat, fix, docs, style, refactor, test, chore)
9 - <scope> is the area of the code affected (optional)
10 - <subject> is a brief summary of the change
11 - <body> provides a more detailed description of the change (optional)
12 - <footer> includes any relevant issue references or breaking change notes (optional)
13 Here are examples of professional git commit messages compared to simple/manual commit messages:
14 1. Simple/Manual Commit Message:
15 ...
16 Added new feature to the project
17 ...
18 Professional Commit Message:
19 ...
20 feat(auth): add user authentication with JWT
21 - Implemented user login and registration endpoints
22 - Added JWT token generation and validation
23 - Updated documentation for authentication process
24 ...
25 2. Simple/Manual Commit Message:
26 ...
27 Fixed bug in the code
28 ...
29 Professional Commit Message:
30 ...
31 fix(api): resolve null pointer exception in user profile endpoint
32 - Added null checks for user data
33 - Updated unit tests to cover edge cases
34 ...
35 3. Simple/Manual Commit Message:
36 ...
37 Updated documentation
```